Complejidad Espacial

Alan Josue Solano Perez

I. Conceptos Fundamentales del Cómputo

La **Teoría de la Complejidad Computacional** tiene como objetivo clasificar los problemas de decisión basándose en la cantidad de recursos que requiere un algoritmo para resolverlos. Para este análisis, se utiliza como modelo teórico la **Máquina de Turing (TM)**. Los dos recursos principales para medir la complejidad son:

1. **Complejidad temporal:** Mide el número de pasos o ciclos que la TM necesita para detenerse y producir una respuesta.
2. **Complejidad espacial:** Mide la cantidad de memoria, o **celdas de la cinta**, que la TM utiliza durante el proceso de cómputo.

Componentes del Espacio y Modelos

El espacio total de memoria se divide conceptualmente:

- **Espacio de entrada:** La parte de la cinta que contiene la descripción del problema, que generalmente es de solo lectura.
- **Espacio de salida:** La parte de la cinta donde se escribe el resultado final.
- **Espacio de trabajo** o **Memoria auxiliar:** El espacio variable que la máquina utiliza para realizar sus cálculos intermedios.

La complejidad espacial se define formalmente mediante la **Función de espacio S(n)**, que acota la cantidad de celdas utilizadas en la memoria auxiliar en función del tamaño **n** de la entrada.

Los problemas se estudian bajo dos paradigmas de la TM: el **Modelo determinista** (la máquina sigue una única trayectoria de cómputo) y el **Modelo no determinista** (la máquina puede explorar múltiples trayectorias simultáneamente, y el problema se acepta si existe al menos una trayectoria de éxito).

II. Clases de Complejidad Espacial y Temporal

Las clases de complejidad espacial se definen por un límite polinómico o logarítmico sobre la función **S(n)**. Las clases temporales se incluyen en esta jerarquía dado que el tiempo de ejecución siempre está limitado por el espacio utilizado (un algoritmo no puede tomar más pasos de los que le permite el espacio para recordar estados).

Clase	Recurso Acotado	Descripción
L (Logspace)	Espacio Logarítmico $\left(\mathbf{O}(\log \mathbf{n})\right)$	Problemas resueltos por una TM determinista con memoria muy estricta, apenas suficiente para almacenar índices o punteros a la entrada.
NL (Nondeterministic Logspace)	Espacio Logarítmico No Determinista	Problemas resueltos por una TM no determinista que utiliza espacio logarítmico. Es la clase clave para la conectividad de grafos.
coNL	Complemento de NL	Clase de problemas cuyo complemento (la versión "negativa") pertenece a NL.
P y **NP**	Tiempo Polinómico	Clases definidas por el **tiempo** de ejecución y verificación (determinista y no determinista), que se incluyen dentro de PSPACE.
PSPACE	Espacio Polinómico $\left(\mathbf{O}\left(\mathbf{n}^{\mathbf{k}}\right)\right)$	Problemas que requieren una cantidad de memoria polinómica, sin importar si el tiempo de ejecución es potencialmente exponencial.
EXPSPACE	Espacio Exponencial $\left(\mathbf{O}\left(\mathbf{2}^{\{\mathbf{n}^{\mathbf{k}}\}}\right)\right)$	Problemas que requieren una cantidad de memoria que crece exponencialmente, siendo la clase de complejidad más alta en esta jerarquía.

III. Relaciones, Problemas y Teoremas Clásicos

La **jerarquía estándar** de inclusión entre las clases de tiempo y espacio es:

$$\mathbf{L} \subseteq \mathbf{NL} \subseteq \mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{PSPACE} \subseteq \mathbf{EXPSPACE}$$

Grandes Teoremas

1. **Teorema de Savitch:** Aunque no está listado en la sección 3C, este teorema es crucial para PSPACE. Establece que el no determinismo no ofrece un ahorro significativo de espacio en el nivel polinómico, demostrando que **PSPACE = NPSPACE** (Savitch, 1970). Esto contrasta con **P** y **NP**, cuya igualdad sigue siendo la pregunta más importante de la informática.
2. **Teorema de Immerman-Szelepcsényi:** Este teorema establece que $\mathbf{NL} = \mathbf{coNL}$. Es un resultado fundamental que demuestra la robustez de las clases de espacio no determinista bajo el complemento, algo que no se sabe si aplica para el tiempo ($\mathbf{NP} ?= \mathbf{coNP}$).

Problemas Clave

- **PATH (Conectividad de grafos):** El problema de decidir si existe un camino entre dos nodos en un grafo dirigido. Es el problema **NL-completo** canónico y un ejemplo clave del poder del no determinismo logarítmico. Un **Algoritmo NL para PATH** es posible al "adivinar" la secuencia de aristas que forman el camino, verificando cada paso con espacio logarítmico.
- **QBF (Satisfacibilidad Booleana Cuantificada):** Este es el problema **PSPACE-completo** por excelencia. Involucra determinar si una fórmula booleana con cuantificadores universales y existenciales es verdadera, lo que a menudo requiere la exploración de un número exponencial de estados, pero solo requiere memoria polinómica.

IV. Técnicas de Análisis y Ejemplos de Algoritmos

Para estudiar la complejidad espacial de manera rigurosa, se utilizan diversas **Técnicas de Análisis**:

- **Conteo de celdas:** El método directo para medir la complejidad espacial, cuantificando el número de celdas utilizadas en la cinta auxiliar.
- **Pila de recursión:** Técnica para medir el espacio consumido por las llamadas recursivas de un algoritmo , esencial para el análisis de algoritmos como el **DFS recursivo** (Búsqueda en Profundidad).
- **Memoria reversible** y **Computación reversible:** Modelos de cómputo donde el proceso puede revertirse lógicamente. Son relevantes en la física de la computación, ya que la reversibilidad está ligada a la minimización de la disipación de energía y, en algunos contextos, a la optimización del espacio.
- **Compresión de recorridos:** Estrategias que buscan representar o simular secuencias largas de cómputo de manera más compacta en la memoria auxiliar.

Ejemplos de Algoritmos por Clase

- **Algoritmos de tiempo bajo (P): Suma de un arreglo** (tiempo lineal,$O(n)$).
- **Algoritmos de espacio bajo (L/NL): Búsqueda binaria** (logarítmico), **Algoritmo NL para PATH**.

- **Algoritmos PSPACE-relevantes:** El **Algoritmo min-max**, usado para determinar estrategias óptimas en juegos con dos jugadores, que a menudo cae en PSPACE si el juego es lo suficientemente complejo (ej. Ajedrez generalizado).

V. Aplicaciones Prácticas y Referentes Históricos

La comprensión de la complejidad espacial y su relación con el tiempo es esencial para la ingeniería de *software* y la informática teórica.

Aplicaciones Prácticas

La teoría tiene impacto directo en:

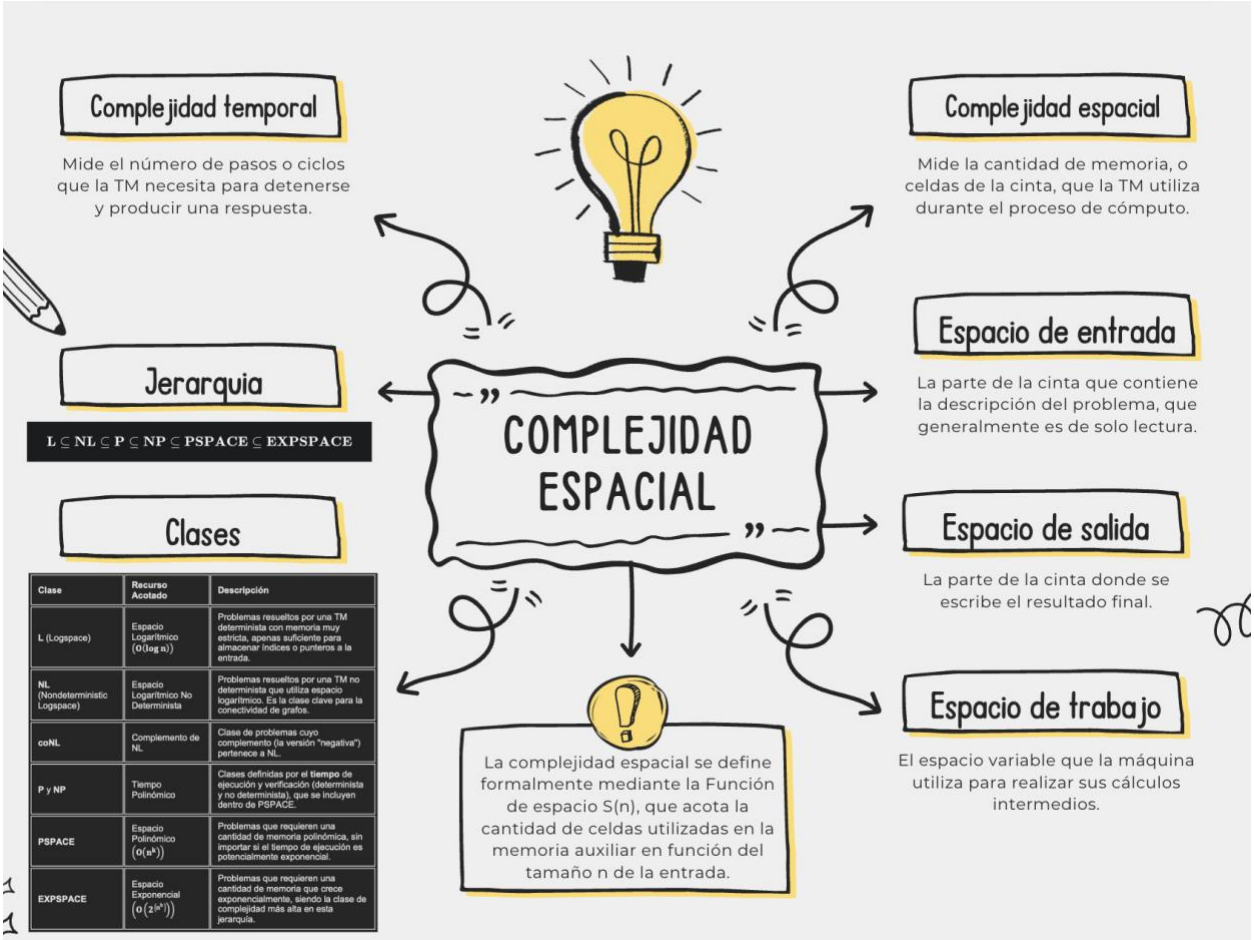
- **Computación en dispositivos móviles** y **Optimización del uso de memoria RAM**, donde la limitación de espacio es crítica.
- Diseño de **Algoritmos para grafos muy grandes** (a través de NL).
- **Criptografía**, que se basa en la intratabilidad (alta complejidad) de ciertos problemas.
- **Verificación formal** (comprobación de la corrección de *hardware* y *software*).
- **Inteligencia artificial y problemas PSPACE-completos**, especialmente en la planificación automática y la resolución de juegos.

Autores y Obras Relevantes

Los principales contribuyentes a esta área, cuyos trabajos son fundamentales para la teoría de la complejidad, incluyen:

- **Michael Sipser**
- **Christos Papadimitriou**
- **John Hopcroft**
- **Rajeev Motwani**
- **Jeffrey Ullman**
- **Charles Bennett** (pionero en computación reversible)
- **Neil Immerman** (codemostrador del teorema NL=coNL)
- **Róbert Szelepcsényi** (codemostrador del teorema NL=coNL)

Mapa Mental Detallado de la Complejidad Computacional



Clase	Recurso Acotado	Descripción
L (Logspace)	Espacio Logarítmico ($O(\log n)$)	Problemas resueltos por una TM determinista con memoria muy estricta, apenas suficiente para almacenar índices o punteros a la entrada.
NL (Nondeterministic Logspace)	Espacio Logarítmico No Determinista	Problemas resueltos por una TM no determinista que utiliza espacio logarítmico. Es la clase clave para la conectividad de grafos.
coNL	Complemento de NL	Clase de problemas cuyo complemento (la versión "negativa") pertenece a NL.
P y NP	Tiempo Polinómico	Clases definidas por el **tiempo** de ejecución y verificación (determinista y no determinista), que se incluyen dentro de PSPACE.
PSPACE	Espacio Polinómico ($O(n^k)$)	Problemas que requieren una cantidad de memoria polinómica, sin importar si el tiempo de ejecución es potencialmente exponencial.
EXPSPACE	Espacio Exponencial ($O(2^{n^k})$)	Problemas que requieren una cantidad de memoria que crece exponencialmente, siendo la clase de complejidad más alta en esta jerarquía.

Referencias

Immerman, N. (1988). Nondeterministic space is closed under complementation. *SIAM Journal on Computing*, *17*(5), 935–938.

Papadimitriou, C. H. (1994). *Computational Complexity*. Addison-Wesley.

Savitch, W. J. (1970). Relationships between non-deterministic and deterministic tape complexities. *Journal of Computer and System Sciences*, *4*(2), 177-192.